

Debugging a full-stack framework like Next.js can feel tricky because your code executes in two entirely different universes: the **Server runtime** (Node.js/Edge) and the **Client environment** (the browser).

If you put a `console.log()` in a Server Component, it prints to your terminal; if you put it in a Client Component, it prints to your browser's Developer Tools.

Here is the ultimate debugging toolkit for tracking down bugs in Next.js.

1. Terminal vs. Browser Console: The Golden Rule

Before hunting for a bug, check where the component is executing.

Component Type	Execution Environment	Where Logs Appear
Server Component (page.tsx default)	Server (Node.js/Edge)	Terminal (where npm run dev is running)
Client Component ('use client')	Server (initial pass) + Browser	Terminal (initial) & Browser DevTools Console
Route Handler (route.ts)	Server (Node.js/Edge)	Terminal
Server Action ('use server')	Server (Node.js/Edge)	Terminal

2. Setting Up Visual Studio Code (VS Code) Breakpoint Debugging

While `console.log` works, using actual breakpoints lets you pause execution, inspect variables, and step through code line by line.

The absolute easiest way to debug Next.js in VS Code is by using the **Auto Attach** feature.

Step-by-Step Breakpoint Setup

1. Open VS Code and open the Command Palette (`Ctrl+Shift+P` on Windows/Linux or `Cmd+Shift+P` on Mac).
2. Search for **"Debug: Toggle Auto Attach"** and select **Only With Flag** or **Always**.
3. Open your VS Code terminal and start your development server with this exact flag:

Bash

```
NODE_OPTIONS='--inspect' npm run dev
```

4. Look at the bottom status bar in VS Code—it will turn orange/blue indicating the debugger is attached.
5. Set a breakpoint by clicking to the left of any line number inside a Server Component or Route Handler, trigger the page reload in your browser, and watch VS Code instantly pause execution right at that line!

3. Demystifying Hydration Errors

Hydration errors are the most common bugs in Next.js. They happen when the initial pre-rendered HTML sent from the server doesn't perfectly match the HTML generated by React when it takes over in the browser.

Common Message: *"Error: Text content does not match server-rendered HTML."*

Why do they happen?

1. **Invalid HTML Nesting:** Placing a `<div>` inside a `<p>` tag, or a `<table>` without a `<tbody>`. Browsers automatically fix this in the DOM, but it breaks React's expectation.
2. **Browser-Only APIs:** Using `window`, `localStorage`, or `document` inside the initial render pass of a client component without waiting for the browser to mount.
3. **Volatile Data:** Rendering changing states like current timestamps (`new Date()`) or random values (`Math.random()`) directly in the markup.

How to debug and fix them:

Look at the browser console. Modern Next.js versions will show a side-by-side diff overlay of exactly what the server expected vs. what the client rendered.

The Fix: Wrap browser-only or volatile code inside a `useEffect` hook so it only triggers *after* hydration is complete.

TypeScript

```
// ❌ WRONG: Will cause hydration mismatch
```

```
export default function TimeWidget() {  
  const time = new Date().toLocaleTimeString();  
  return <div>Current Time: {time}</div>;  
}
```

```
// CORRECT: Safely synchronizes post-hydration
'use client';

import { useState, useEffect } from 'react';

export default function TimeWidget() {
  const [time, setTime] = useState<string | null>(null);

  useEffect(() => {
    setTime(new Date().toLocaleTimeString());
  }, []);

  if (!time) return <div>Loading...</div>; // Render a safe placeholder first
  return <div>Current Time: {time}</div>;
}
```

4. Debugging Server Actions ('use server')

Because Server Actions behave like quiet background POST requests, tracking failures requires looking closely at network payloads.

1. Open your browser **DevTools** and go to the **Network** tab.
2. Trigger the Server Action in your UI.
3. Look for a network request where the Status is 200 or 500 and the type is fetch.
4. Click the request, select the **Payload** tab to see exactly what arguments your frontend sent to the server, and check the **Response** tab to see what the server sent back.

5. Analyzing Production Bundle Sizes

Is your Next.js application taking too long to load in production? You can visually profile your compiled application to find heavy dependencies.

Install the official Next.js Bundle Analyzer package:

Bash

```
npm install @next/bundle-analyzer
```

Wrap your configuration in next.config.js:

JavaScript

```
// next.config.js
```

```
const withBundleAnalyzer = require('@next/bundle-analyzer')({  
  enabled: process.env.ANALYZE === 'true',  
})
```

```
module.exports = withBundleAnalyzer({  
  // Your standard next config options here  
})
```

Run a specialized build command:

Bash

```
ANALYZE=true npm run build
```

This will automatically launch an interactive, color-coded map grid in your browser showing exactly how many kilobytes each NPM package adds to your application, allowing you to instantly swap heavy dependencies out for lighter alternatives.